

Algorithmes et Pensée Computationnelle

Fonctions, mémoire et exceptions

Le but de cette séance est d'approfondir vos connaissances en programmation. Au terme de cette séance, vous serez capable de :

- définir une fonction et l'utiliser dans un programme,
- utiliser des bibliothèques contenant des fonctions prédéfinies,
- connaître la portée d'une variable,
- comprendre comment fonctionne une pile d'exécution (call stack),
- gérer des exceptions.

1 Opérations arithmétiques

Question 1: (🕒 3 minutes) [Java et Python] Calculs (multiplication)

Créez deux variables `facteur_1` (= 11) et `facteur_2` (= 3). Multipliez la première variable par la deuxième et stockez le résultat dans une nouvelle variable `produit`. Vous pouvez afficher les différentes variables pour voir leurs valeurs. Vous pouvez répéter l'exercice avec l'addition et la soustraction.

💡 Conseil

L'opérateur de multiplication est le `*`, celui de l'addition est le `+` et celui de soustraction est le `-`.

>_ Solution

Python :

```
1 if __name__ == "__main__":
2     facteur_1 = 11
3     facteur_2 = 3
4     produit = facteur_1*facteur_2
5     print(facteur_1)
6     print(facteur_2)
7     print(produit)
```

Java :

```
1 public class Question1 {
2     public static void main(String[] args) {
3         int facteur_1 = 11;
4         int facteur_2 = 3;
5         int produit = facteur_1*facteur_2;
6         System.out.println(facteur_1);
7         System.out.println(facteur_2);
8         System.out.println(produit);
9     }
10 }
```

Question 2: (🕒 10 minutes) [Java et Python] Calculs (division)

Créez deux variables `nb_bonbons` avec pour valeur 11 et `nb_personnes` avec pour valeur 3. Divisez la première variable par la deuxième et stockez le résultat dans une nouvelle variable `bonbons_personnes`. Pour finir, calculez le nombre de bonbons restants via l'opérateur `%` (modulo) et stockez le résultat dans une nouvelle variable `reste`. Vous pouvez afficher les différentes variables pour voir leurs valeurs.

Conseil

- Attention : en Python il existe deux opérateurs de division, / effectue une division classique, tandis que // effectue une division avec arrondi inférieur.
- En Java, lorsque les deux opérandes sont de type int, l'opérateur / effectue une division entière. Si au moins l'un des deux opérandes est de type float ou double, il effectue une division en virgule flottante.
- L'opérateur % (modulo) permet de calculer le reste d'une division. Par exemple, 10%2=0.

>_ Solution

Python :

```
1 if __name__ == "__main__":
2     nb_bonbons = 11
3     nb_personnes = 3
4     bonbons_personnes = nb_bonbons // nb_personnes
5     reste = nb_bonbons % nb_personnes
6     print (nb_bonbons)
7     print (nb_personnes)
8     print (bonbons_personnes)
9     print (reste)
```

Java :

```
1 public class Question2 {
2     public static void main(String[] args) {
3         int nb_bonbons = 11;
4         int nb_personnes = 3;
5         int bonbons_personnes = nb_bonbons / nb_personnes;
6         int reste = nb_bonbons % nb_personnes;
7         System.out.println(nb_bonbons);
8         System.out.println(nb_personnes);
9         System.out.println(bonbons_personnes);
10        System.out.println(reste);
11    }
12 }
```

Question 3: (🕒 5 minutes) [Java et Python] Calculs (incrémentation / décrémentation)

Gardez vos variables de l'exercice précédent, augmentez la valeur de nb_bonbons de 1 et diminuez celle de nb_personnes de 1.

Conseil

Vous pouvez utiliser les opérateurs += et -= en Python, et les opérateurs ++ (incrémentation) et -- (décrémentation) en Java.

>_ Solution

Python :

```
1 nb_bonbons = 11
2 nb_personnes = 3
3 nb_bonbons += 1
4 nb_personnes -= 1
5 bonbons_personnes = nb_bonbons // nb_personnes
6 reste = nb_bonbons % nb_personnes
7 print (nb_bonbons)
8 print (nb_personnes)
9 print (bonbons_personnes)
10 print (reste)
```

Java :

```
1 public class Question3 {
2     public static void main(String[] args) {
3
4         int nb_bonbons = 11;
5         int nb_personnes = 3;
6         nb_bonbons ++;
7         nb_personnes --;
8         int bonbons_personnes = nb_bonbons / nb_personnes;
9         int reste = nb_bonbons % nb_personnes;
10        System.out.println(nb_bonbons);
11        System.out.println(nb_personnes);
12        System.out.println(bonbons_personnes);
13        System.out.println(reste);
14    }
15 }
```

2 Variables et Fonctions

Question 4: (🕒 5 minutes) [Java et Python] Les fonctions (fonctions basiques)

Définissez une fonction nommée `ping()` qui, lorsqu'elle est appelée, affiche `pong`.

💡 Conseil

Référez-vous aux diapositives du cours pour la création et l'appel des fonctions.

>_ Solution

Python :

```
1 def ping() :  
2     print("pong")  
3 if __name__ == "__main__":  
4     ping()
```

Java :

```
1 public class Question4 {  
2     static void Ping() {  
3         System.out.println("pong");  
4     }  
5  
6     public static void main(String[] args) {  
7         Ping();  
8     }  
9 }
```

Question 5: (🕒 5 minutes) [Java et Python] Les Fonctions (Fonction multiplication)

Définissez une fonction nommée `multiplicateur()` qui prend deux arguments *multiple1* et *multiple2*, les multiplie et retourne le résultat. Stockez le résultat de `multiplicateur(2,3)` dans une variable *resultat* et affichez-la.



Conseil

- Référez-vous aux diapositives du cours pour la création et l'appel des fonctions.
- Pour retourner une valeur au lieu de l'afficher, utilisez le mot-clé `return` (pour Python et Java).

>_ Solution

Python :

```
1 def multiplicateur(multiple1, multiple2):
2     return multiple1 * multiple2
3
4
5 if __name__ == "__main__":
6     resultat = multiplicateur(2, 3)
7     print(resultat)
```

Java :

```
1 public class Question5 {
2     public static int multiplicateur(int multiple1, int
multiple2) {
3         return multiple1*multiple2;
4     }
5
6     public static void main(String[] args) {
7         int resultat = multiplicateur(2, 3);
8         System.out.println(resultat);
9     }
10 }
```

Question 6: (🕒 5 minutes) [Java et Python] Les Fonctions (Fonctions Aire et Périmètre)

Définissez deux fonctions nommées `aire()` et `perimetre()` qui prennent un argument (`rayon`) et renvoient respectivement l'aire et le périmètre d'un cercle. Stockez les résultats dans des variables `aire` et `perimetre` et affichez le contenu de ces variables.



Conseil

- Référez-vous aux diapositives du cours pour la création et l'appel des fonctions.
- Pour retourner une valeur au lieu de l'imprimer, utilisez le mot-clé `return` (pour Python et Java).
- Pour rappel, le périmètre d'un cercle s'obtient en utilisant la formule $P = 2 * \pi * r$ et l'aire s'obtient en utilisant la formule $A = r^2 * \pi$.

>_ Solution

Python :

```
1 import math
2
3 def aire(rayon):
4     return (rayon**2)*math.pi
5
6 def perimetre(rayon):
7     return 2*math.pi*rayon
8
9 if __name__ == '__main__':
10     rayon = 10
11     aire = aire(rayon)
12     perimetre = perimetre(rayon)
13     print("L'aire d'un cercle de rayon {} est égale à
14           {}".format(rayon, aire))
15     print("Le périmètre d'un cercle de rayon {} est égal à
16           {}".format(rayon, perimetre))
```

Java :

```
1 public class Question6 {
2
3     static double aire(int rayon){
4         return Math.pow(rayon, 2)*Math.PI;
5     }
6
7     static double perimetre(int rayon){
8         return 2*Math.PI*rayon;
9     }
10
11     public static void main(String[] args) {
12         int rayon = 5;
13         double aire = aire(rayon);
14         double perimetre = perimetre(rayon);
15         System.out.println("L'aire d'un cercle de rayon
16                             "+rayon+" est égale à "+aire);
17         System.out.println("Le périmètre d'un cercle de rayon
18                             "+rayon+" est égal à "+perimetre);
19     }
20 }
```

Question 7: (🕒 5 minutes) [Python] Portée des variables

Qu'affiche le programme suivant ?

```
1 x = 2
2
3 def fonction():
4     x = 3
5
6     def fonction2():
7         global x
8         print("x=" + str(x))
9
10     fonction2()
```

```

11     print("x=" + str(x))
12
13 print(fonction())

```

💡 Conseil

- Le mot-clé `global` permet d'accéder aux variables globales (définies à l'extérieur de la fonction).
- Soyez attentif au type des éléments retournés par chacune des fonctions.

>_ Solution

`x = 2` est une variable globale (en dehors de la fonction), alors que `x = 3` est une variable locale. La fonction imbriquée `fonction2` fait appel à la variable globale, avec le mot-clé `global` devant. Lors du deuxième appel à `print`, il n'y a pas le mot clé `global` donc la variable locale est utilisée.

La solution est donc :

```

x=2
x=3
None

```

Question 8: (🕒 10 minutes) [Java] Portée des variables

Qu'affiche le programme suivant ?

```

1 public class Question8 {
2     static int a = 0;
3     public static void f() {
4         a += 2;
5         System.out.println("a = " + a);
6     }
7
8     public static void g() {
9         int b = 3;
10        System.out.println("a = " + a);
11        System.out.println("b = " + b);
12    }
13
14    public static void main(String[] args) {
15        f();
16        g();
17        //System.out.println("b = " + b);
18    }
19 }

```

Que se passerait-il si on décommente la ligne 17 (c'est-à-dire si l'on enlève les `//`) ?

💡 Conseil

Tenir compte de l'endroit où chaque variable est définie ainsi que des opérations réalisées sur chaque variable.

>_ Solution

```
— a = 2
  a = 2
  b = 3
```

— On obtiendra une erreur de compilation du programme car la variable `b` est créée à l'intérieur de la méthode `g()` et n'existe donc qu'au sein de celle-ci. La variable `b` n'étant pas définie en dehors de la méthode `g()`, celle-ci ne peut pas faire l'objet d'un appel en dehors de la méthode `g()`.

3 Gestion des exceptions

Question 9: (🕒 5 minutes) [Python] **Types d'erreurs** Qu'affiche le programme suivant ?

```
1 value1 = "Algorithms"
2 value2 = 4
3
4 try:
5     size = len(value1)
6     result = size/value2
7     print(f"Le résultat de la division est: {result}")
8
9 except Exception as error:
10    print("On ne peut pas effectuer l'opération")
11
12 try:
13     result = value1/2
14     print(f"Le résultat de la division est: {result}")
15
16 except TypeError as error:
17    print("On ne peut pas diviser une chaîne de caractères")
```

💡 Conseil

Utilisée sur une chaîne de caractères, la fonction `len()` renvoie le nombre de caractères de la chaîne.

>_ Solution

Le résultat de la division est: 2.5
On ne peut pas diviser une chaîne de caractères.

Question 10: (🕒 5 minutes) [Python] **Types d'erreurs** Qu'affiche le programme suivant ?

```
1 value1 = 4
2 value2 = ""
3
4 try:
5     count = len(value2)
6     result = value1/count
7 except ZeroDivisionError as error:
8     print("Nous ne pouvons pas diviser un nombre par 0")
```

💡 Conseil

La chaîne de caractères vide est représentée par ""

>_ Solution

Nous ne pouvons pas diviser un nombre par 0.

Question 11: (🕒 5 minutes) [Python] **Types d'erreurs** Qu'affiche le programme suivant ?

```
1 value1 = "Algorithms"
2 value2 = 4
3
4
5 try:
6     decimal = float(value2)
7 except ValueError as error:
8     print("Nous ne pouvons pas convertir un entier en décimal")
9
10 finally:
11     try:
12         value2 = int(value1)
13     except ValueError as error:
14         print("Nous ne pouvons pas convertir une chaîne de caractères
    en nombre")
```

💡 Conseil

- La fonction `float()` permet de convertir une variable en nombre à virgule flottante.
- La fonction `int()` permet de convertir une variable en nombre entier.

>_ Solution

`float(value2)` est valide car on transforme un nombre entier en un float. Aucune erreur ne va être lancée.

`int(value1)` n'est pas valide si `value1` contient une chaîne qui ne représente pas un entier, comme `"abc"`. En revanche, une chaîne comme `"42"` peut être convertie en entier.

La chaîne `"Algorithms"` provoque ainsi une erreur avec le message suivant :
Nous ne pouvons pas convertir une chaîne de caractères en nombre

Question 12: (🕒 5 minutes) [Java] **Gestion d'erreurs** Le programme suivant est incorrect. Que devez-vous modifier pour qu'il fonctionne correctement ?

💡 Conseil

Référez-vous à la diapositive 31 du cours de cette semaine.

```
1 public class Question12 {
2
3     public static void division(int a, int b) throws
        ArithmeticException {
4         if (b == 0) {
```

```

5         throw new ArithmeticException();
6     } else {
7         float result = a / b;
8         System.out.println("Le résultat de la division de " + a +
9         "/" + b + " = " + result);
10    }
11
12    public static void main(String args[]) {
13        int value1 = 2;
14        int value2 = 4;
15        try {
16            division(value1, value2);
17            value2 = 0;
18            division(value1, value2);
19        } catch (IndexOutOfBoundsException err) {
20            System.out.println("Nous ne pouvons pas effectuer une
21            division par 0");
22        }
23    }

```

>_ Solution

À l'intérieur de la fonction `division`, lorsque `b` est égal à 0, le programme lève une exception de type `ArithmeticException`. Mais dans le `main`, le bloc `catch` est configuré pour intercepter l'exception `IndexOutOfBoundsException`. Pour corriger cette erreur, une des solutions est de remplacer `IndexOutOfBoundsException` par `ArithmeticException`.

Question 13: ⌚ 5 minutes [Java] Gestion d'erreurs

1. Qu'affiche le programme suivant, en supposant que l'utilisateur saisit les entrées suivantes ?

- Entrez le premier entier : 12
- Entrez le deuxième entier : 2
- Entrez l'opération (+, -, *, /): *

💡 Conseil

Consultez [ce document](#) sur le `switch` en Java. Référez-vous à la diapositive 23 du cours.

2. Que devez-vous modifier pour qu'il fonctionne ?

```

1 import java.util.Scanner;
2
3 public class Calculator {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Entrez le premier entier: ");
8         double num1 = sc.nextInt();
9
10        System.out.print("Entrez le deuxième entier: ");
11        double num2 = sc.nextInt();
12
13        System.out.print("Entrez l'opération (+, -, *, /): ");
14        char op = sc.next().charAt(0);

```

```

15
16     sc.close();
17
18     switch (op) {
19         case '+':
20             System.out.println(num1 + num2);
21         case '-':
22             System.out.println(num1 - num2);
23         case '*':
24             System.out.println(num1 * num2);
25         case '/':
26             System.out.println(num1 / num2);
27         default:
28             throw new IllegalArgumentException("L'opération n'est
pas valide");
29     }
30 }
31 }

```

>_ Solution

```

Entrez le premier entier: 12
Entrez le deuxième entier: 2
Entrez l'opération (+, -, *, /): *
24.0
6.0
Exception in thread "main" java.lang.IllegalArgumentException:
L'opération n'est pas valide
at Calculator.main(Calculator.java:28)

```

>_ Solution

Il faut simplement ajouter un `break;` après l'affichage du résultat d'une opération.

Question 14: (🕒 5 minutes) Qu'affiche le programme suivant, en supposant que l'utilisateur exécute le programme de la manière suivante dans le terminal ?

```
python3 question14.py 9
```

💡 Conseil

Référez-vous à la diapositive 15 du cours.

```

1 import sys
2 if __name__ == '__main__':
3     balance = 20
4     try:
5         balance = balance - sys.argv[1]
6         solde_post = balance > 0
7         match (solde_post):
8             case True:
9                 print('Le solde de votre compte est positif.')
10            case False:
11                print('Le solde de votre compte est négatif ou zéro.')
12                raise ValueError('Le compte ne peut pas être débité')
13    except Exception as e:

```

```
14     print("Une erreur est survenue")
15 finally:
16     print(f"Valeur finale de balance: {balance}")
```

>_ Solution

```
Une erreur est survenue
Valeur finale de balance: 20
```

Cette erreur est due à la soustraction d'une chaîne de caractères (`sys.argv[1]`) d'un nombre, ce qui n'est pas définie.

Question 15: (🕒 5 minutes) Laquelle des affirmations suivantes est **fausse** concernant la différence entre une Error (erreur) et une Exception (exception) en Java ?

💡 Conseil

Référez-vous à la diapositive 36 du cours.

- (A) Les erreurs sont généralement causées par l'environnement dans lequel l'application s'exécute, tandis que les exceptions sont généralement causées par l'application elle-même.
- (B) Les erreurs sont non vérifiées (*unchecked*) et n'ont pas besoin d'être déclarées dans la clause `throws` d'une méthode, tandis que les exceptions peuvent être soit vérifiées (*checked*), soit non vérifiées (*unchecked*).
- (C) Les erreurs devraient généralement être interceptées et gérées par l'application afin d'assurer une récupération fluide, tandis que les exceptions peuvent être récupérables ou non.
- (D) Les erreurs indiquent généralement des problèmes graves qu'une application raisonnable ne devrait pas essayer d'intercepter.
- (E) `RuntimeException` sont des exceptions non vérifiées (*unchecked*).

>_ Solution

(C) L'affirmation C est fausse car elle inverse l'approche habituelle. Les erreurs (telles que `OutOfMemoryError`, `StackOverflowError`) ne doivent généralement pas être détectées et gérées par les applications, car elles indiquent des problèmes graves qui ne peuvent généralement pas être résolus. Les exceptions, en revanche, sont conçues pour être détectées et gérées par l'application lorsque cela est approprié, car elles représentent souvent des conditions récupérables (telles que `ArithmeticException` ou `IllegalArgumentException`).